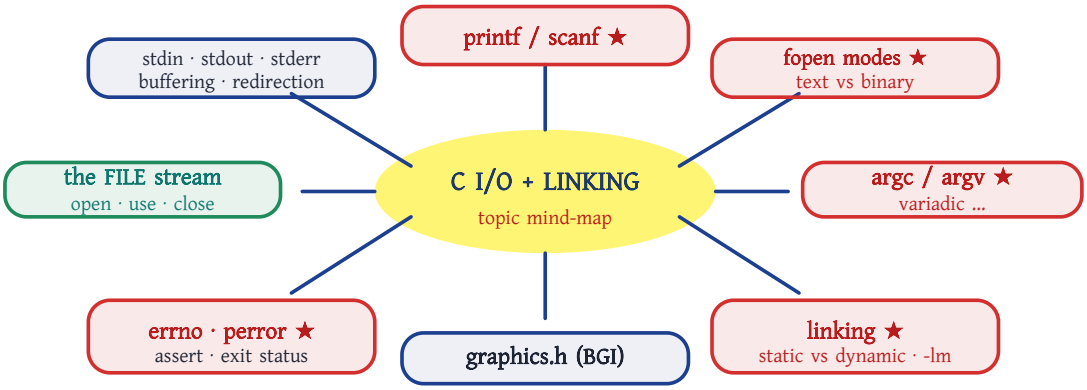
I/O, Files, Errors & Linking in C

printf/scanf · fopen · fseek · stdarg · argc/argv · errno · linking · exam-ready in 11 sheets



· fseek / ftell / rewind · the feof() trap · va_start / va_arg / va_end · library headers
· preprocessor → compiler → assembler → linker → loader · extern and multi-file programs

↑ the whole lesson on one page: red = high-yield, green = the idea everything else is built on

Syllabus coverage

The structured-programming syllabus line reads "**Input and Output: standard input and output, formatted input and output, file access; Variable length argument list; Command line parameters; Error Handling; Graphics; Linking; Library functions**". Data types, functions, pointers and structures have their own lessons.

High-yield list

① scanf: %f = float, %lf = double
② size_t (strlen, sizeof) → %zu
③ fopen fails ⇒ returns NULL
④ "w" truncates · "a" always writes at the end
⑤ ./prog one two ⇒ argc = 3, argv[3] = NULL
⑥ never va_arg(ap, float) — use double
⑦ perror writes to stderr
⑧ static link copies only the needed modules; -lm = libm

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

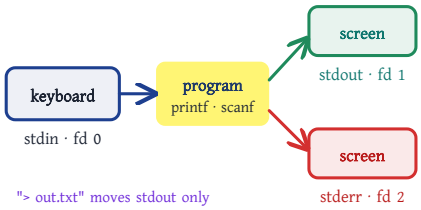
Contents

✓ Streams & printf ... p. 2
✓ scanf & character I/O ... p. 3
✓ File access & modes ... p. 4
✓ Random access & EOF ... p. 5
✓ Variadic functions & argc/argv ... p. 6
✓ Error handling ... p. 7
✓ Graphics & library functions ... p. 8
✓ Compiling & linking ... p. 9 · Night p. 10 · Practice p.

What the whole lesson is for: A C program talks to the outside world only through **streams** and the **return values** of library calls, and becomes runnable only after the **linker** joins it to those libraries — so the whole lesson is **streams in, checks on, linker last.**

1 Standard Streams & Formatted Output (printf)

Definition: A **stream** is C's uniform view of any source or destination of bytes — keyboard, screen, file, pipe — reached through a `FILE *`. Every hosted C program starts with **three streams already open**, declared in **<stdio.h>**: **stdin** (keyboard), **stdout** (screen; line-buffered on a terminal, fully buffered into a file) and **stderr** (screen, **unbuffered** — shows at once).



two outputs that look the same on screen — but they are **separate streams**

Exam trap: (i) **stderr** is not a log file: it goes to the screen too, but `prog > out.txt` redirects only stdout, so errors still show. (ii) Text left in the stdout buffer is **lost if the program crashes** — `fflush(stdout)` forces it out.

Spec.	Argument	Prints
%d %i	int	-42
%u	unsigned int	42
%ld %lld	long, long long	123456789
%f	**double (float is promoted)**	3.140000
%Lf	long double	3.140000
%e %g	double: exponent / shorter	3.14e+00 · 3.14

Spec.	Argument	Prints
%c	char (as int)	A
%s	char * string	Dhaka
%p	void * pointer	0x7ffd...
%x %X %o	unsigned: hex / octal	ff · FF · 377
%zu	**size_t**	5
%%	nothing	a literal %

Anatomy: % [flags] [width] [.precision] [length] letter:

- left-justify · **+** show the sign · **0** pad with zeros · **#** 0x for %x, 0 for %o

width = a **minimum**, never truncates; ***** takes it from an argument

.precision: digits after the point (%f) · max chars (%s)

length: hh, h, l, ll, L, z — the argument's size

printf returns the number of characters printed (negative on error): `printf("%d", printf("Hello"))` prints **Hello5**.

Worked example ✍️ predict the output (_ = a space): `"%5d|"`, 42 → **___42|** · `"%-5d|"`, 42 → **42___|** · `"%05d"`, 42 → **00042** · `"%+d"`, 42 → **+42** · `"%.2f"`, 3.14159 → **3.14** · `"%8.3f|"`, 3.14159 → **___3.142|** · `"%#x %X %o"`, 255 ×3 → **0xff FF 377** · `"%.3s"`, "Bangladesh" → **Ban** · `"%2d"`, 12345 → **12345**.

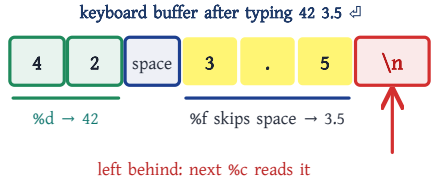
Exam trap: (i) In **printf**, %f serves float *and* double — a float is promoted to double; the split matters only in scanf. (ii) `printf("%d", 3.5)` is **undefined**. (iii) `strlen` returns **size_t**: print it with **%zu**, not %d or %lu.

Memory trick: "**Flags, Width, Dot, Length, Letter**" — always in that order. "**Minus moves left, zero fills, plus shows.**"

2 Formatted Input (scanf) & Character / Line I/O

Definition: **scanf** reads characters from stdin, converts them according to the format and **stores each result through a pointer** — so every non-array argument needs **&**. It returns the **number of items assigned**, or **EOF** if input fails before the first conversion. %d, %f and %s skip leading white space; **%c does not**.

Type	printf	scanf	Why they differ
float	%f	%f	gets a **float *** — 4 bytes
double	%f (also %lf since C99)	**%lf**	a **double ***: 8 bytes; %f fills only 4
long double	%Lf	%Lf	capital L, never lowercase
short / long	%hd / %ld	%hd / %ld	size must match the pointer
char	%c	%c, or **" %c"**	the space skips the leftover '\n'
string	%s	%s with **no &**	an array name is an address; stops at a space



`scanf("%d %f", &n, &x)` stops before the Enter key

Function	Does
getchar() / putchar(c)	one character in / out; getchar returns an **int** so EOF fits
gets(s)	removed in C11 — no size limit, buffer overflow
fgets(s, n, stdin)	at most **n - 1** chars, **keeps the '\n'**, adds '\0'
puts(s) / fputs(s, fp)	puts adds a '\n'; fputs adds nothing
getch() / getche()	a key without Enter — **conio.h**, DOS/Turbo C only, not standard

Worked example ✍ what does scanf return?: `int a = 0, b = 0, r = scanf("%d %d", &a, &b);` **(a)** input **10 20** → r = **2**, a = 10, b = 20. **(b)** input **10 abc** → r = **1**, b stays 0 and "abc" **stays in the buffer** for the next read. **(c)** input **abc** → r = **0**. **(d)** end of input (Ctrl+Z / Ctrl+D) → r = **EOF**. Safe loop: `while (scanf("%d", &x) == 1) sum += x;`

Three scanf tools worth knowing:

%9s — width limit: at most 9 chars into `char s[10]`

%[^\n] — scanset: a whole line, spaces included

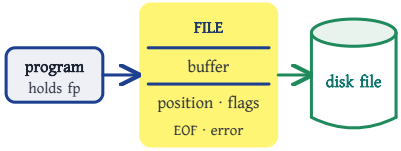
%*d — read and **discard**; not counted in the return

Exam trap: (i) `scanf("%d", x)` without & passes a value as an address — undefined behaviour, usually a crash. (ii) `char c = getchar();` can never equal EOF where char is unsigned — keep it **int**. (iii) `fflush(stdin)` is undefined in standard C. (iv) %s reads **one word** only.

Memory trick: "**printf takes values, scanf takes addresses.**" And "**lf = long float = double**": in scanf the **l** is the only thing standing between a float and a double.

3 File Access: FILE *, fopen Modes, Text vs Binary

Definition: A file is used through a **pointer to a FILE** structure (declared in <stdio.h>) that holds the **buffer**, the **current position** and the **end-of-file and error flags**. Every file job has three steps: **open** with `fopen(name, mode)`, which **returns NULL on failure**; **process**; **close** with `fclose(fp)`, which flushes the buffer and returns 0 (or EOF on failure).



fp = fopen(...) — check it for NULL
fclose(fp) flushes the buffer to disk

the program holds only a pointer; the FILE does the bookkeeping

Mode	If the file exists	If it is missing	Allows
"r"	opened at the start	**NULL**	read
"w"	**truncated to 0**	created	write
"a"	kept; **every** write goes to the end	created	append
"r+"	opened, **not** truncated	**NULL**	read + write
"w+"	truncated	created	write + read
"a+"	kept; reads anywhere, writes at end	created	read + append

Add **b** for **binary**: "rb", "wb", "ab", "r+b" (= "rb+"), "w+b", "a+b". C11 adds **x** ("wx"): fail instead of overwriting. Note `printf(…)` ≡ `fprintf(stdout, …)`.

Level	Read	Write	Signal at end / error
character	fgetc(fp), getc(fp)	fputc(c, fp), putc(c, fp)	EOF
line / string	fgets(s, n, fp)	fputs(s, fp)	NULL · EOF
formatted	fscanf(fp, fmt, ...)	fprintf(fp, fmt, ...)	count or EOF · negative
block (binary) ★	fread(ptr, size, n, fp)	fwrite(ptr, size, n, fp)	items done, **< n** on end or error

```
/* copy in.txt to out.txt */
FILE *in  = fopen("in.txt", "r");
FILE *out = fopen("out.txt", "w");
if (in == NULL || out == NULL) {
    perror("fopen");
    return 1;
}
int c;              /* int, not char */
while ((c = fgetc(in)) != EOF)
    fputc(c, out);
```

Text vs binary: In **text mode** the library may translate line ends — on Windows '\n' is written as "\r\n" and read back as '\n'. **Binary mode** moves bytes exactly. On Linux/UNIX the two modes are identical. `fprintf(fp, "%d", 12345)` writes **5 characters**; `fwrite(&n, sizeof n, 1, fp)` writes **sizeof(int) bytes** (4 on most systems).

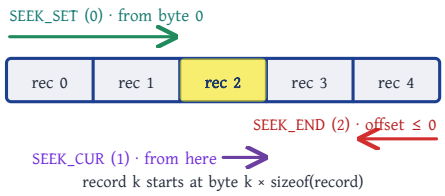
```
fclose(in);
fclose(out);
```

Exam trap: (i) **"w" destroys** the old contents; use "a" to keep them. (ii) **"r+"** never creates a file. (iii) fopen signals failure with **NULL**, not EOF or -1. (iv) Forgetting fclose can **lose buffered data**. (v) In "a"/"a+" a write lands at the end even after fseek.

Memory trick: "**r reads, w wipes, a adds — and + gives the other half.**" "**b means bytes.**" Only the two r-modes fail on a missing file; every w- and a-mode creates it.

4 Random Access: fseek, ftell, rewind & the EOF Rules

Definition: Every open stream has a **file position indicator** — the byte offset where the next read or write happens. Sequential I/O moves it forward by itself; **random access** moves it directly with `fseek(fp, offset, whence)`, so record *k* of a binary file can be read without reading the *k* - 1 before it.



three origins, one offset — the whole of random access

Call	Effect · return
fseek(fp, off, whence)	move to origin + off · **0** on success
ftell(fp)	current offset as **long** · -1L on error
rewind(fp)	back to 0 **and** clears the error flag · void
feof(fp)	non-zero once a read has **already hit** the end
ferror(fp) / clearerr(fp)	error flag set? / reset both flags
fflush(fp)	write an output buffer out now
remove(n) / rename(o, n)	delete / rename a file · 0 on success

Worked example ✍️ a binary file of student records: `struct student { int roll; char name[16]; float cgpa; };` — 4 + 16 + 4 = **24 bytes** (no padding needed here; padding rules are in the structures lesson). **(a)** File size: `fseek(fp, 0, SEEK_END); long n = ftell(fp);` → n = 240 ⇒ 240 ÷ 24 = **10 records**. **(b)** Read the **5th** record (k = 4): `fseek(fp, 4 * sizeof(struct student), SEEK_SET)` → byte **96**, then `fread(&s, sizeof s, 1, fp)` returns 1. **(c)** Now `ftell(fp)` = 96 + 24 = **120**. **(d)** Update it in place: open with **"rb+"**, `fseek(fp, -24L, SEEK_CUR)`, then `fwrite(&s, sizeof s, 1, fp)`. **(e)** The last record: `fseek(fp, -24L, SEEK_END)` → byte 216.

```
/* WRONG: the last line prints twice */
while (!feof(fp)) {
    fgets(line, 80, fp);
    fputs(line, stdout);
}
/* RIGHT: test the read itself */
while (fgets(line, sizeof line, fp) != NULL)
    fputs(line, stdout);
```

EOF is a value, not a byte: **EOF** is a macro in <stdio.h> for a **negative int** (usually -1) that input functions return when nothing is left. No EOF character is stored in the file; a file simply ends. That is why **fgetc returns int**: it must fit all 256 byte values *and* EOF.

Exam trap: (i) **feof() does not look ahead**: it turns true only after a read has failed, so a `while (!feof(fp))` loop runs once too often. (ii) Byte arithmetic with ftell needs **binary** mode. (iii) Between a write and a read, call fseek or fflush.

Memory trick: "**SET, CUR, END = 0, 1, 2 — start, here, end.**" And "**read first, then ask**": test the value fgetc/fgets/fread returned; ask feof() only to learn *why* it stopped.

5 Variable-Length Argument Lists (stdarg.h)

Definition: A **variadic function** takes a variable number of arguments, declared with an ellipsis **...** after **at least one named parameter**: `int printf(const char *fmt, ...);`. The macros of **<stdarg.h>** walk the unnamed arguments one by one; the function itself has **no way to know how many** arrived.

```
#include <stdarg.h>
int sum(int count, ...) {
    va_list ap;
    int total = 0;
    va_start(ap, count);
    for (int i = 0; i < count; i++)
        total += va_arg(ap, int);
    va_end(ap);
    return total;
}
sum(4, 10, 20, 30, 40)   → 100
sum(2, 5, 15)            → 20
```

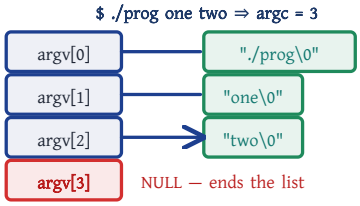
Macro / type	Job
va_list ap	the cursor over the arguments
va_start(ap, last)	start after the **last named** parameter
va_arg(ap, T)	fetch the next one as type T
va_copy(d, s)	duplicate the cursor (C99)
va_end(ap)	clean up — one per va_start

Exam trap: Unnamed arguments undergo the **default promotions**: char, short → **int**; float → **double**. So **va_arg(ap, float)** and va_arg(ap, char) are undefined — ask for double / int. Reading past the last argument is also undefined.

How the callee learns the count: a **count** argument (sum above) · a **sentinel** at the end (NULL, -1) · a **format string** — printf counts the % signs. Get the count wrong and va_arg reads garbage.

6 Command-Line Parameters (argc, argv)

Definition: `int main(int argc, char *argv[])` (or `char **argv`) receives the words typed after the command. **argc** = how many words, **the program name included**; **argv[0]** = the program name; argv[1] … argv[argc - 1] = the arguments, **always as strings**; and **argv[argc] = NULL**.



argv is an array of pointers to strings

Worked example ✍ `$ ./sum 10 20 30`: argc = **4**; argv[1] is the **string** "10", so the sum needs a conversion: `for (i = 1; i < argc; i++) s += atoi(argv[i]);` → 60. **(b)** `$ ./prog "hello world"` → argc = **2**: the shell's quotes make one argument. **(c)** `$ ./prog` alone → argc = 1, argv[1] = NULL: test `argc < 2` first. **(d)** atoi("12ab") silently gives 12 — **strtol** can report errors.

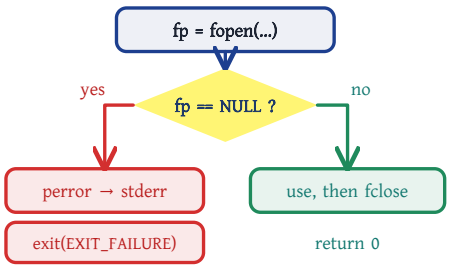
Exam trap: (i) argc is never 0 in practice — the name counts. (ii) `argv[1] + argv[2]` adds **pointers**, not numbers. (iii) The names argc/argv are only convention; the **types** are what the standard fixes.

Memory trick: "**argc counts the name; argv ends in NULL.**" For stdarg: "**start, arg, arg, ..., end**" — and "**small things grow**": char → int, float → double.

7 Error Handling: Return Values, errno, perror, assert, exit

Definition: C has **no exceptions**. A library function reports failure through its **return value** and, for many, by storing a code in **errno**; the program must **check**, report on **stderr**, and end with a **failure exit status**. Nothing happens automatically if the check is skipped.

Call	Failure looks like
fopen, malloc, calloc, fgets	**NULL**
scanf, fscanf	fewer items than asked, or **EOF**
fgetc, getchar, fclose, fputs	**EOF**
printf, fprintf	a **negative** value
fseek / ftell	non-zero / **-1L**
fread, fwrite	fewer than **n** items
sqrt(-1) · strtol overflow	errno = **EDOM** · **ERANGE**



check, tell, leave — every library call

```c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>
FILE *fp = fopen("data.txt", "r");
if (fp == NULL) {
    perror("data.txt");
    fprintf(stderr, "code %d: %s\n",
            errno, strerror(errno));
    exit(EXIT_FAILURE);
}
```

errno · perror · strerror: **errno** (<errno.h>) is set to a positive code by a failing call and **never reset to 0** by the library. ISO C defines only **EDOM, ERANGE, EILSEQ**; ENOENT etc. are POSIX. **perror("data.txt")** prints **data.txt: No such file or directory** to stderr; **strerror(errno)** (<string.h>) returns that text.

assert, exit, abort — how a program ends:

assert(expr): if false, prints expression, file, line to stderr, then **abort()**

#define NDEBUG before <assert.h> ⇒ every assert **disappears**

exit(status): flushes and closes streams, runs atexit() handlers

return n; in main ≡ **exit(n)** · EXIT_SUCCESS 0 · EXIT_FAILURE ≠ 0

abort(): abnormal end, no atexit handlers. The shell reads the status as **$?** / **%ERRORLEVEL%**.

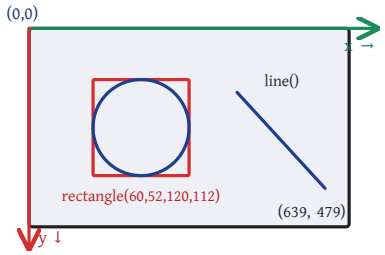
Worked example ✍️ division by zero: Integer `a / b` with b = 0 is **undefined behaviour** — no return code, no errno, usually a crash — so **test first**: `if (b == 0) { fputs("divide by zero\n", stderr); return EXIT_FAILURE; }`. (IEEE 754 floats give 1.0 / 0.0 = **inf**.)

Exam trap: (i) errno is meaningful only **right after** a call that failed. (ii) **assert** is not input validation — it vanishes in a release build. (iii) Report errors on **stderr**, not stdout. (iv) **0 = success**, non-zero = failure — not the reverse.

Memory trick: "**Check, tell, leave.**" Check the return value, tell stderr (perror), leave with EXIT_FAILURE. "**Assert for bugs, if for users.**"

8 Graphics in C (graphics.h, Turbo C)

Definition: **graphics.h** is Borland's **BGI** (Borland Graphics Interface) for Turbo C/C++ under DOS — **not part of ISO C** (gcc has no such header). The screen is a pixel grid with the **origin (0, 0) at the top-left** and **y growing downward**; in VGA high mode (640 × 480, 16 colours) `getmaxx()` = 639 and `getmaxy()` = 479.



screen coordinates: y points **down**

Function	Does
initgraph(&gd, &gm, path)	graphics mode; **gd = DETECT**
graphresult() · closegraph()	**grOk** (0) · back to text
putpixel(x,y,c) · line(x1,y1,x2,y2)	a pixel · a line
circle(x,y,r) · rectangle(l,t,r,b)	radius · **two corners**
setcolor(c) · setbkcolor(c)	BLACK 0 · RED 4 · WHITE 15
setfillstyle + floodfill(x,y,b)	fill up to border b
outtextxy(x,y,s) · cleardevice()	text · clear screen

```
#include <graphics.h>
#include <conio.h>    /* getch() */
int main(void) {
  int gd = DETECT, gm;
  initgraph(&gd, &gm, "C:\\TC\\BGI");
  if (graphresult() != grOk) return 1;
  setcolor(RED);  rectangle(100,100,200,200);
  setcolor(BLUE); circle(150, 150, 50);
  outtextxy(110, 220, "Graphics Example");
  getch(); closegraph(); return 0;
}
```

Exam trap: (i) **getch()** is in **conio.h**, not graphics.h — it just holds the picture until a key is pressed. (ii) rectangle takes **two corners**, not width and height. (iii) A larger y is **lower**.

9 Standard Library Functions & Headers

Definition: The **C standard library** is precompiled code shipped with the compiler. A **header declares** (prototypes, macros, types); the **library archive defines** (the machine code the linker adds). C89 had **15** standard headers, C99 **24**, C11 **29**.

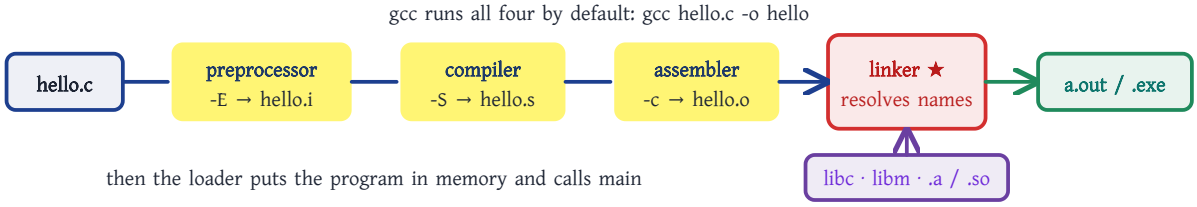
Header	High-yield members
<stdio.h>	printf, scanf, fopen, fgets, fread, perror, EOF, FILE, NULL
<stdlib.h>	malloc, calloc, realloc, free, atoi, strtol, **rand, srand, exit, abs**, qsort, system
<string.h>	strlen, strcpy, strcat, strcmp, strchr, strstr, memcpy, memset, strerror
<math.h>	sqrt, pow, fabs, ceil, floor, sin, cos, log, log10, exp — all return **double**
<ctype.h> · **<time.h>**	isalpha, isdigit, toupper, tolower · time, clock, difftime
<limits.h> · **<stddef.h>**	INT_MAX, CHAR_BIT · size_t, NULL, offsetof

Worked example ✍ outputs: strlen("Dhaka") = **5**, sizeof("Dhaka") = **6** · pow(2, 10) = **1024.000000** · floor(−2.1) = **−3.0** · `rand() % 6 + 1` = a die roll after `srand(time(NULL))`.

Memory trick: "**abs in stdlib, fabs in math.**" "**Header declares, library defines**" — only the linker brings the code.

10 From Source to Executable: Compiling & Linking

Definition: Building a C program takes four stages, and the **linker** is the last: it combines object files and libraries into one executable by **resolving symbols** — matching every use of an external name with exactly one definition — and **relocating** addresses. The **loader** then places the executable in memory and starts it.



stop after any stage with its flag: -E, -S or -c

Stage	In → out	What it does · typical error
Preprocessor	.c → .i	#include, #define, #if; strips comments (preprocessor lesson) · missing header file
Compiler	.i → .s	syntax and type checks, assembly code · missing semicolon, type mismatch
Assembler	.s → .o / .obj	machine code with **unresolved** external symbols
Linker ★	.o + libs → exe	resolves and relocates · **undefined reference**, multiple definition

Aspect	Static linking	Dynamic (shared) linking
Library file	.a (Linux) · .lib (Windows)	.so (Linux) · .dll (Windows)
When bound	at link time: **only the needed object modules** are copied in	at load time by the dynamic linker (or run time: dlopen, LoadLibrary)
Size · memory	larger file; each program has its own copy	smaller file; **one copy shared** in memory
Updating the library	relink every program	replace the .so/.dll once
Risk	none at run time — self-contained	missing or wrong version ("DLL hell")

```
/* util.h */ extern int counter;
             int add(int a, int b);
/* util.c */ int counter = 0;
             int add(int a, int b)
             { counter++; return a + b; }
/* main.c */ #include "util.h"
gcc -c main.c      → main.o
gcc -c util.c      → util.o
gcc main.o util.o -o prog
gcc prog.c -o prog -lm   (libm)
```

extern & linkage: **extern** declares a name defined **elsewhere**; the definition must appear in exactly **one** file. **static** at file scope gives **internal linkage**: the name is invisible to the linker and to other files (storage classes lesson). **-lm** links libm, the maths library; put it **after** the source files.

Exam trap: (i) <math.h> gives only prototypes; "**undefined reference to sqrt**" is a **linker** error cured by -lm. (ii) Static linking does **not** copy the whole library.

Memory trick: "**Please Compile And Link**" — Preprocess, Compile, Assemble, Link; the flags **-E -S -c** stop after the first three. "**Static = suitcase, dynamic = borrow on arrival.**"

Night-Before-the-Exam Sheet

Fact sheet:

stdin 0 · stdout 1 · stderr 2

stderr is **unbuffered**

%d int · %u unsigned · %ld long

%c char · %s string · %p void *

%x hex · %o octal · %e exponent

size_t → %zu

printf %f = float *and* double

scanf: %f float, %lf double

% flags width .prec length letter

width is a minimum, never cuts

printf returns chars printed

scanf returns items assigned / EOF

scanf needs &, but not for arrays

gets removed in C11 → fgets

fgets keeps '\n', reads n − 1

getchar returns **int**

fopen fails ⇒ **NULL**

"w" truncates · "a" appends

"r", "r+" never create a file

b = binary · x = exclusive (C11)

fread/fwrite return items done

SEEK_SET 0 · CUR 1 · END 2

size = fseek END + ftell

rewind = seek 0 + clear error

feof true only **after** a failed read

EOF = negative int, usually -1

va_start · va_arg · va_end

float → double, char → int

argc counts the program name

argv[argc] = NULL

perror → stderr

errno: EDOM · ERANGE · EILSEQ

assert off with NDEBUG

EXIT_SUCCESS 0 · EXIT_FAILURE

getch() is conio.h

BGI: (0,0) top-left, y down

-E · -S · -c · link

static .a/.lib · dynamic .so/.dll

-lm links the maths library

abs stdlib · fabs math

Top 12 one-liners for MCQs

1. printf and scanf are declared in **<stdio.h>**
2. `printf("%d", printf("Hello"))` ⇒ **Hello5**
3. `"%8.3f"` of 3.14159 ⇒ **___3.142**
4. Read a double: **scanf("%lf", &d)**
5. fopen on a missing file in "r" ⇒ **NULL**
6. Record 5 of 24-byte records starts at byte **96**
7. `./prog one two` ⇒ argc = **3**
8. Variadic macros live in **<stdarg.h>**
9. perror("x") ⇒ "x: <reason>" on **stderr**
10. assert false ⇒ message + **abort()**
11. undefined reference to sqrt ⇒ add **-lm**
12. Shared library is bound at **load time**

Top traps: %f for a double in scanf · %lu or %d for size_t · & forgotten in scanf · `char c = getchar()` · **while (!feof(fp))** · "w" when the data had to be kept · fopen checked against EOF · va_arg(ap, float) · argc said to exclude the program name · errors on stdout · assert to validate input · getch() put in graphics.h · **static linking said to copy the whole library**.

All tricks together: "Flags, Width, Dot, Length, Letter" · "printf takes values, scanf takes addresses" · "lf = long float = double" · "r reads, w wipes, a adds; + gives the other half; b means bytes" · "SET, CUR, END = 0, 1, 2" · "read first, then ask" · "start, arg, arg, ..., end; small things grow" · "argc counts the name; argv ends in NULL" · "Check, tell, leave" · "Assert for bugs, if for users" · "abs in stdlib, fabs in math" · "Please Compile And Link" · "Static = suitcase, dynamic = borrow on arrival".

✓ Practice MCQs (exam pattern, 10 marks)

1. Which conversion reads a **double** with scanf?
   (a) %f (b) %d
   (c) %lf (d) %Lf

2. What does fopen() return when it cannot open the file?
   (a) a NULL pointer (b) EOF
   (c) -1 (d) an empty FILE

3. A program with `int main(int argc, char *argv[])` is run as `./prog one two`. argc is:
   (a) 1 (b) 2
   (c) 4 (d) 3

4. Which mode opens a file for writing and **erases** any existing contents?
   (a) "a" (b) "w"
   (c) "r+" (d) "a+"

5. `printf("%5.2f|", 3.14159);` prints (_ = a space):
   (a) 3.14| (b) _3.14|
   (c) 3.14159| (d) 00003.14|

6. va_start, va_arg and va_end are defined in:
   (a) <stdio.h> (b) <stdlib.h>
   (c) <stdarg.h> (d) <conio.h>

7. gcc reports "undefined reference to sqrt". The fix is to:
   (a) add -lm when linking (b) add #include <math.h>
   (c) compile with -c (d) compile with -E

8. Which is true of **feof(fp)**?
   (a) it predicts the next read will fail (b) it returns the EOF character
   (c) it is declared in conio.h (d) it is true only after a read has hit the end

9. perror() writes its message to:
   (a) stdout (b) a log file
   (c) stderr (d) stdin

10. In **static** linking the linker:
    (a) loads the library at run time (b) copies only the needed object modules
    (c) makes the executable smaller (d) needs the .dll at run time

Answer key → 1-c · 2-a · 3-d · 4-b · 5-b · 6-c · 7-a · 8-d · 9-c · 10-b

Written question: Write a C program that takes a file name on the command line, reads the integers in that file and prints their sum, handling every error properly. Explain argc/argv and each check you make. 10

1. **Arguments** — `int main(int argc, char *argv[])`; argc counts the program name, so one file name means **argc == 2**, and argv[1] is the name. If `argc != 2`, print a usage line to stderr and `return EXIT_FAILURE;`. 2
2. **Open** — `FILE *fp = fopen(argv[1], "r");` and `if (fp == NULL) { perror(argv[1]); return EXIT_FAILURE; }` — perror adds the reason from errno, on stderr. 2
3. **Read** — `long sum = 0; int x; while (fscanf(fp, "%d", &x) == 1) sum += x;` — the loop tests the **return value** (not feof), so it stops on end of file and on bad data; then `if (ferror(fp))` report a read error. 3
4. **Finish** — `fclose(fp); printf("Sum = %ld\n", sum); return EXIT_SUCCESS;` — status `0` tells the shell it worked. 1
5. **Explain** — argv holds strings, argv[argc] is NULL; every library call is checked because C has no exceptions: NULL from fopen, the item count from fscanf, the flag from ferror. 2

Before the exam ✓

- ✓ Can predict printf output for any width, precision and flag
- ✓ Can state all six fopen modes and what each does to an existing file
- ✓ Can compute fseek offsets for record k
- ✓ Can name the four build stages and contrast static with dynamic linking